

UNIT-2(CONTROL FLOW AND LOOPS)

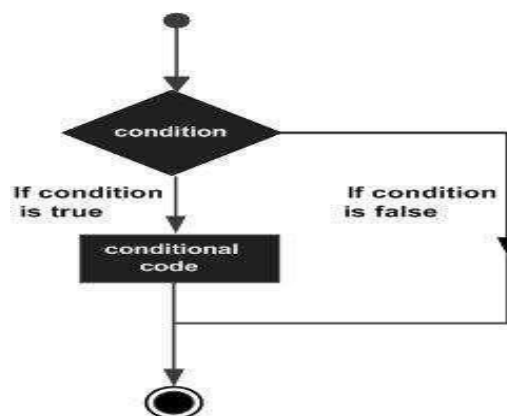
Control flow statements:

There comes situations in real life when we need to make some decisions and based on these decisions, we decide what should we do next.

Similar situations arise in programming also where we need to make some decisions and based on these decisions we will execute the next block of code.

Decision-making statements in programming languages decide the direction of the flow of program execution.

In Python, if, if-else elif statement is used for decision making.



Python programming language assumes any non-zero and non-null values as True, and if it is either zero or null, then it is assumed as False value.

Statement	Description
if statements	if statement consists of a boolean expression followed by one or more statements.
if...else statements	if statement can be followed by an optional else statement , which executes when the boolean expression is FALSE.
nested if statements	You can use one if or else if statement inside another if or else if statement(s).

The *if* Statement

if statement is the most simple decision-making statement. It is used to decide whether a certain statement or block of statements will be executed or not.

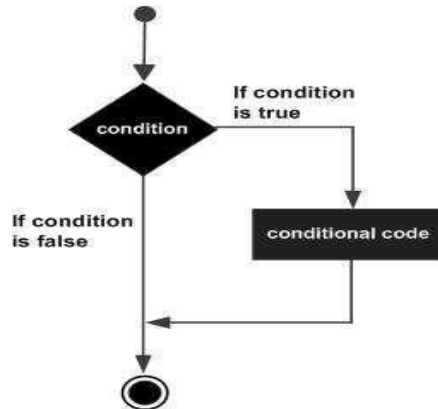
i.e if a certain condition is true then a block of statement is executed otherwise not..

Syntax:

```
if condition:
    statements
```

First, the condition is tested. If the condition is True, then the statements given after colon (:) are executed. We can write one or more statements after colon (:).

FLOWCHART:



Example:

```

a=10
b=15
if a < b:
    print "B is big"
    print "B value is",b
  
```

```

B is big
15
  
```

The *if... else* statement

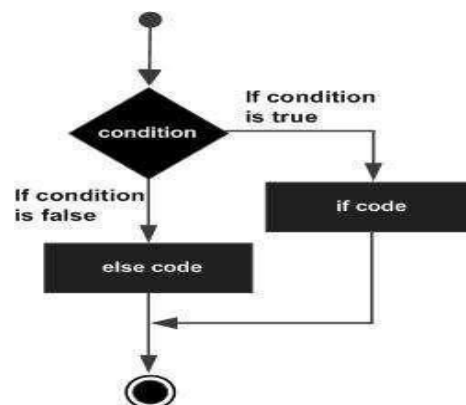
An **else** statement can be combined with an **if** statement. An **else** statement contains the block of code that executes if the conditional expression in the if statement resolves to 0 or a FALSE value.

The *else* statement is an optional statement and there could be at most only one **else** statement following **if**.

Syntax:

```

if condition:
    statement(s)
else:
    statement(s)
  
```



Example:

```
a=48 b=34
if a < b:
    print( "B is big" )
    print( "B value is", b)
else:
    print ( "A is big")
    print ( "A value is", a)
    print( "END")
```

Output:

```
A is big
A value is 48
END
```

Q) Write a program for checking whether the given number is even or not.

Program:

```
a=int(input("Enter a "))
if a%2==0:
    print ("a is EVEN num")
else:
    print ("a is NOT EVEN Number")
```

Output-1:

```
Enter a : 56
a is EVEN num
```

Output-2:

```
Enter a value: 27
a is NOT EVEN Number
```

The *elif* Statement

The **elif** statement allows you to check multiple expressions for True and execute a block of code as soon as one of the conditions evaluates to True.

Similar to the **else**, the **elif** statement is optional. However, unlike **else**, for which there can be at most one statement, there can be an arbitrary number of **elif** statements following an **if**.

Syntax:

```
if condition1:
    statement(s)
elif condition2:
    statement(s)
else:
    statement(s)
```

Example:

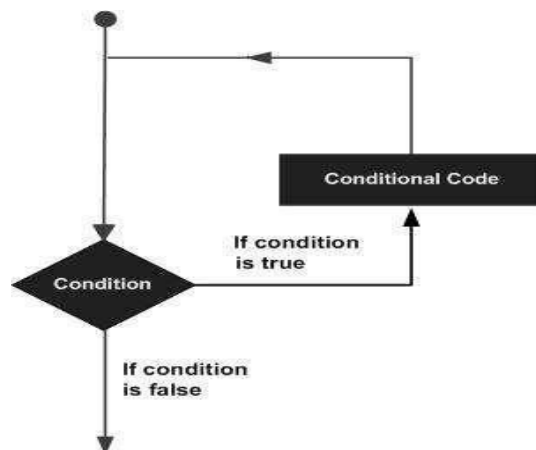
```
a=20
b=10
c=30
if a >= b and a >= c:
    print( "a is big")
elif b >= a and b >= c:
    print( "b is big")
else:
    print ("c is big")
```

Output:

c is big

Decision Loops

- In general, statements are executed sequentially: The first statement in a function is executed first, followed by the second, and so on. There may be a situation when you need to execute a block of code several number of times.
- A loop statement allows us to execute a statement or group of statements multiple times. The following diagram illustrates a loop statement:
-



Python programming language provides following types of loops to handle looping requirements.

Loop Type	Description
while loop	Repeats a statement or group of statements while a given condition is TRUE. It tests the condition before executing the loop body.
for loop	Executes a sequence of statements multiple times and abbreviates the code that manages the loop variable.
nested loops	You can use one or more loop inside any another while, for loop.

The while Loop

A **while** loop statement in Python programming language repeatedly executes a target statement as long as a given condition is True.

Syntax

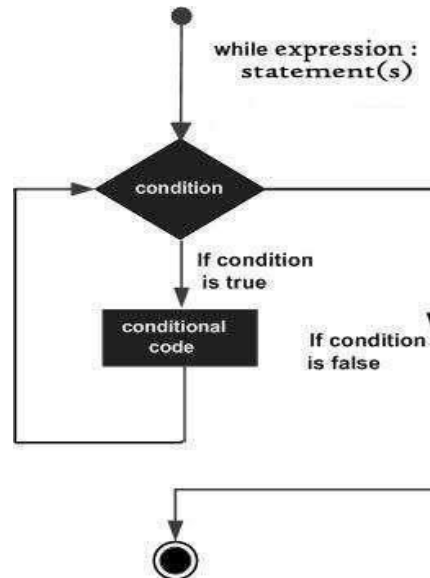
The syntax of a **while** loop in Python programming language is:

```
while expression:
    statement(s)
```

Here, **statement(s)** may be a single statement or a block of statements.

The **condition** may be any expression, and true is any non-zero value. The loop iterates while the condition is true. When the condition becomes false, program control passes to the line immediately following the loop.

In Python, all the statements indented by the same number of character spaces after a programming construct are considered to be part of a single block of code. Python uses indentation as its method of grouping statements.



Example-1:

```
i=1
while i < 4:
    print i
    i+=1
    print "END"
```

Example-2:

```
i=1
while i < 4:
    print i
    i+=1
    print "END"
```

Output-1:

```
1
END
2
END
3
END
```

Output-2:

```
1
2
3
END
```

Q) Write a program to display factorial of a given number.

Program:

```
n=int(input("Enter the number: "))
f=1
while n>0:
    f=f*n
    n=n-1
print ("Factorial is",f)
```

Output:

Enter the number: 5

Factorial is 120

2.2.2.for loop:

The *for* loop is useful to iterate over the elements of a sequence. It means, the *for* loop can be used to execute a group of statements repeatedly depending upon the number of elements in the sequence. The *for* loop can work with sequence like string, list, tuple, range etc.

The syntax of the *for* loop is given below:

```
for var in sequence:
    statement (s)
```

The first element of the sequence is assigned to the variable written after „for“ and then the statements are executed.

Next, the second element of the sequence is assigned to the variable and then the statements are executed second time.

In this way, for each element of the sequence, the statements are executed once. So, the *for* loop is executed as many times as there are number of elements in the sequence.

Example-1:

```
for i in range(1,5):
    print( i)
    print (“END”)
```

Example-2:

```
for i in range(1,5):
    print (i)
    print (“END”)
```

Output-1:

```
1
END
2
END
3
END
4
END
```

Output-2:

```
1
2
3
4
END
```

Q) Write a program to display the factorial of given number.

Program:

```
n=int(input("Enter the number: "))
f=1
for i in range(1,n+1):
    f=f*i
print( "Factorial is",f)
```

OUTPUT:

```
Enter the number: 5
Factorial is 120
```

Nested Loop:

It is possible to write one loop inside another loop. For example, we can write a for loop inside a while loop or a for loop inside another for loop. Such loops are called “nested loops”.

Example-1:

```
for i in range(1,6):
    for j in range(1,6):
        print (j,end='')
    print (" ")
```

```
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
```

Example-2:

```
for i in range(1,6):
    for j in range(1,6):
        print ("*",end='')
    print ""
```

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

1) Write a program for print given number is prime number or not using for loop. Program:

```
n=int(input("Enter the value"))

count=0

for i in range(2,n):

    if n%i==0:

        count+=1

        break

if count==0:

    print ("Prime Number")

else:

    print ("Not Prime Number")
```

Output:

Enter n value: 17

Prime Number

2) Write a program print Fibonacci series and sum the even numbers. Fibonacci series is 1,2,3,5,8,13,21,34,55

```
n=int(input("Enter n value "))
f0=1
f1=2
print( f0,f1)
for i in range(1,n-1):
    f2=f0+f1
    print (f2)
    f0=f1
    f1=f2
```

Output:

Enter n value 10

1 2 3 5 8 13 21 34 55 89

3) Write a program to print n prime numbers and display the sum of prime numbers.

Program:

```
n=int(input("Enter the range: "))
sum=0
for num in range(1,n+1):
    for i in range(2,num):
        if (num % i) == 0:
            break
        else:
            print( num)
            sum += num
print ("\nSum of prime numbers is",sum)
```

Output:

```
Enter the range: 21
1 2 3 5 7 11 13 17 19
Sum of prime numbers is 78
```

4) Write a program to print sum of digits.

Program:

```
n=int(input("Enter the number: "))
sum=0
while n>0:
    r=n%10
    sum+=r
    n=n/10
print ("sum is",sum)
```

Output:

```
Enter the number: 123456789
sum is 45
```

5) Write a program to print given number is Armstrong or not.

Program:

```
n=int(input("Enter the number: "))
sum=0
t=n
while n>0:
    r=n%10
    sum+=r*r*r
    n=n/10
if sum==t:
    print("ARMSTRONG")
else:
    print ("NOT ARMSTRONG")
```

Output:

Enter the number: 153
ARMSTRONG

BREAK:

The break statement in Python terminates the current loop and resumes execution at the next statement, just like the traditional break found in C.

The most common use for break is when some external condition is triggered requiring a hasty exit from a loop. The break statement can be used in both while and for loops.

Example:

```
for letter in 'Python':    # First Example
    if letter == 'h':
        break
    print ('Current Letter :', letter)
```

OUTPUT

Current Letter : P
Current Letter : y
Current Letter : t

CONTINUE:

The continue statement in Python returns the control to the beginning of the while loop. The continue statement rejects all the remaining statements in the current iteration of the loop and moves the control back to the top of the loop.

The continue statement can be used in both while and for loops.

EXAMPLE 1 :

```
for letter in 'Python':    # First Example
    if letter == 'h':
        continue
    print ('Current Letter :', letter)
```

OUTPUT

Current Letter : P
Current Letter : y
Current Letter : t
Current Letter : o
Current Letter : n

Example2:

```
var = 10
while var > 0:
    var = var -1
    if var == 5:
        continue
    print( 'Current variable value :', var)
print( "Good bye!")
```

OUTPUT

```
Current variable value : 10
Current variable value : 9
Current variable value : 8
Current variable value : 7
Current variable value : 6
Current variable value : 4
Current variable value : 3
Current variable value : 2
Current variable value : 1
Good bye!
```

2.2.5.PASS:

The pass statement in Python is used when a statement is required syntactically but you do not want any command or code to execute.

The pass statement is a null operation; nothing happens when it executes. The pass is also useful in places where your code will eventually go, but has not been written yet

EXAMPLE

```
for letter in 'Python':
    if letter == 'h':
        pass
    print( 'This is pass block')
    print( 'Current Letter :', letter)

print ("Good bye!")
```

OUTPUT

```
Current Letter : P
Current Letter : y
Current Letter : t
This is pass block
Current Letter : h
Current Letter : o
Current Letter : n
Good bye!
```

assert Keyword in Python

In python, **assert** keyword helps in achieving this task. This statement takes as input a boolean condition, which when returns true doesn't do anything and continues the normal flow of execution, but if it is computed to be false, then it raises an AssertionError along with the optional message provided.

Syntax : assert condition, error_message(optional)

EXAMPLE:

```
a = 4
b = 0
print("The value of a / b is : ")
assert b != 0
print(a / b)
```

OUTPUT:

```
The value of a / b is :
-----
-----
-
AssertionError
```